Experiment No.: 03
Title: Implementation of Sparse Table

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 01**

Aim: Implementation of Sparse Table using Range Minimum Query.

Resources needed: Text Editor, C/C++ IDE

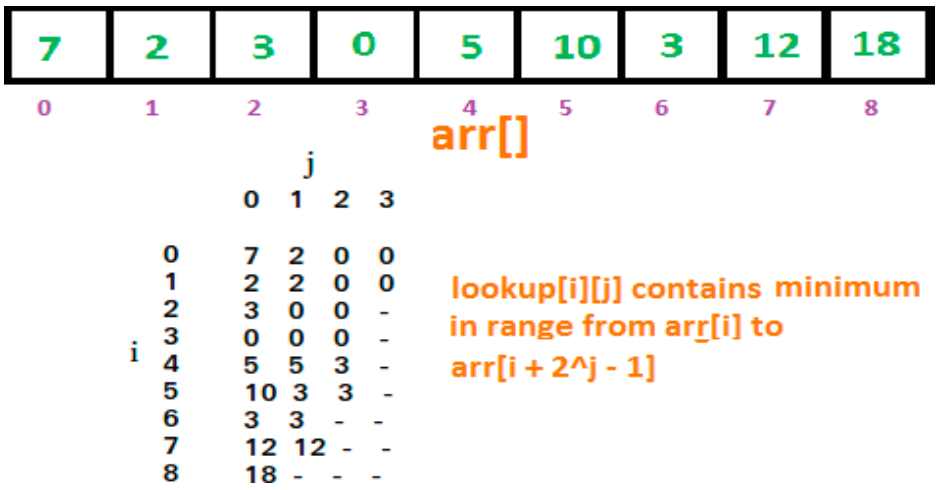
Theory:

A Sparse Table is a powerful data structure used for solving Range Queries efficiently when updates are not required. It excels at answering idempotent queries (queries where repeated application does not change the result), such as Range Minimum Query (RMQ), Range Maximum Query (RMaxQ), and Greatest Common Divisor (GCD) queries.

Sparse table concept is used for fast queries on a set of static data (elements do not change). It does preprocess so that the queries can be answered efficiently.

Steps for creating Sparse table:

- Initialize lookup[i][0] = arr[i] for every $0 \leq i < n$.
- For each j from 1 up to $\lfloor \log_2 n \rfloor$, and for each i from 0 to n – 2^j:
- Compute lookup[i][j] as the minimum of the two halves:
- the interval beginning at i of length 2^(j–1) (lookup[i][j–1])
- the interval beginning at i + 2^(j–1) of length 2^(j–1) (lookup[i + 2^(j–1)][j–1])
- To answer a query on the range [L, R]:
- Let $k = \lfloor \log_2(R - L + 1) \rfloor$.
- The minimum over [L, R] is min(lookup[L][k], lookup[R – 2^k + 1][k]).



Approach for creating Range Minimum Query Using Sparse Table-

The idea is to precompute the minimum values for all subarrays whose lengths are **powers of two** and store them in a table so that any range-minimum query can be answered in constant time. We build a 2D array lookup where lookup[i][j] holds the minimum of the subarray

starting at i with length 2^j (j varies from to Log n where n is the length of the input array). For example lookup[0][3] contains minimum of range [0, 7] (starting with 0 and of size 23) solution.

How to fill this lookup or sparse table?

The idea is simple, fill in a bottom-up manner using previously computed values. We compute ranges with current power of 2 using values of lower power of two. Each entry for length 2^j is derived by combining two overlapping subarrays of length 2^(j–1) that were computed in the previous step. For example, to find a minimum of range [0, 7] ,Range size is a power of 3, we can use the minimum of following two.

a) Minimum of range [0, 3] (Range size is a power of 2)

b) Minimum of range [4, 7] (Range size is a power of 2) Based on above example, below is formula,

// Minimum of single element subarrays is same // as the only element. lookup[i][0] = arr[i] // If lookup[0][2] <= lookup[4][2], // then lookup[0][3] = lookup[0][2] If lookup[i][j-1] <= lookup[i+2j-1][j-1] lookup[i][j] = lookup[i][j-1] // If lookup[0][2] > lookup[4][2], // then lookup[0][3] = lookup[4][2] Else lookup[i][j] = lookup[i+2j-1][j-1]

Limitation

The primary limitations of a sparse table for range minimum queries (RMQ) relate to **immutability** of the underlying data and its requirements for the type of query function.

No updates: If any element in the array is updated between queries, the entire sparse table, which requires ($n\log n$) construction time, must be recomputed.

Not suitable for non-idempotent operations: Operations like range sum queries are not idempotent, as including an element in the sum twice would yield an incorrect result.

Memory usage: Sparse tables require ($n\log n$) space for storage, which can be a problem if you have a very large array (n).

Construction time: The time required to precompute the table is ($n\log n$), which is not always feasible for extremely large datasets.

Procedure:

You are given an integer array arr of length n and an integer q denoting the number of queries. Each query consists of two indices L and R ($0 \le L \le R < n$), and asks for the minimum value in the subarray arr[L…R].

Input format

```
arr[] = [ 7, 2, 3, 0, 5, 10, 3, 12, 18 ]
queries[][] = [ [0, 4], [4, 7], [7, 8] ]
```

Output format

```
0 3 12
```

Use case Implemetation:

The implementation's use case was heatwave monitoring on daily weather data. It used a Sparse Table to answer range minimum queries on `temperature_2m_max` after preprocessing, so you can quickly find the coolest day in any date window and compare it against heatwave thresholds.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 32
#define MAXLOG 6

typedef struct {
    char date[16];
    double tmax;
} DayRecord;

DayRecord records[] = {
    {"2000-01-01", 19.9},
    {"2000-01-02", 20.0},
    {"2000-01-03", 20.1},
    {"2000-01-04", 19.8},
    {"2000-01-05", 21.4},
    {"2000-01-06", 23.2},
    {"2000-01-07", 24.1},
    {"2000-01-08", 22.7},
    {"2000-01-09", 25.6},
    {"2000-01-10", 26.3}
};

double st[MAXN][MAXLOG];
int lg2[MAXN];
int n = 10;

void build_sparse_table(double arr[], int n) {
    int i, j;

    lg2[1] = 0;
    for (i = 2; i <= n; i++) {
        lg2[i] = lg2[i / 2] + 1;
    }

    for (i = 0; i < n; i++) {
        st[i][0] = arr[i];
    }

    for (j = 1; (1 << j) <= n; j++) {
        for (i = 0; i + (1 << j) <= n; i++) {
            double left = st[i][j - 1];
            double right = st[i + (1 << (j - 1))][j - 1];
            st[i][j] = (left < right) ? left : right;
        }
    }
}
```

```c
double query_rmq(int L, int R) {
    int len = R - L + 1;
    int k = lg2[len];
    double left = st[L][k];
    double right = st[R - (1 << k) + 1][k];
    return (left < right) ? left : right;
}

int main(void) {
    double temps[MAXN];
    int i;

    for (i = 0; i < n; i++) {
        temps[i] = records[i].tmax;
    }

    build_sparse_table(temps, n);

    printf("Heatwave monitoring series based on temperature_2m_max\n");
    printf("Using Sparse Table RMQ for minimum temperature queries\n\n");

    printf("Sample data:\n");
    for (i = 0; i < n; i++) {
        printf("%2d  %s  %.1f C\n", i, records[i].date, records[i].tmax);
    }

    printf("\nRMQ examples:\n");

    int L1 = 0, R1 = 4;
    double ans1 = query_rmq(L1, R1);
    printf("Minimum temperature from %s to %s = %.1f C\n",
           records[L1].date, records[R1].date, ans1);

    int L2 = 3, R2 = 8;
    double ans2 = query_rmq(L2, R2);
    printf("Minimum temperature from %s to %s = %.1f C\n",
           records[L2].date, records[R2].date, ans2);

    int L3 = 5, R3 = 9;
    double ans3 = query_rmq(L3, R3);
    printf("Minimum temperature from %s to %s = %.1f C\n",
           records[L3].date, records[R3].date, ans3);

    return 0;
}
```

Output

```
Heatwave monitoring series based on temperature_2m_max
Using Sparse Table RMQ for minimum temperature queries
Sample data:
 0  2000-01-01  19.9 C
 1  2000-01-02  20.0 C
 2  2000-01-03  20.1 C
 3  2000-01-04  19.8 C
 4  2000-01-05  21.4 C
 5  2000-01-06  23.2 C
```

```
 6  2000-01-07  24.1 C
 7  2000-01-08  22.7 C
 8  2000-01-09  25.6 C
 9  2000-01-10  26.3 C

RMQ examples:
Minimum temperature from 2000-01-01 to 2000-01-05 = 19.8 C
Minimum temperature from 2000-01-04 to 2000-01-09 = 19.8 C
Minimum temperature from 2000-01-06 to 2000-01-10 = 22.7 C

=== Code Execution Successful ===
```

Conclusion:

The Sparse Table was successfully implemented for Range Minimum Queries. It efficiently preprocesses static data and answers queries in constant time, demonstrating its usefulness for fast minimum-value retrieval without frequent updates.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

1. Antti Laaksonen, “Guide to Competitive Programming”,Springer,2018
2. Gayle Laakmann McDowell,” Cracking the Coding Interview”,CareerCup LLC,2015
3. Steven S. Skiena Miguel A. Revilla,”Programming challenges, The Programming Contest Training Manual”, Springer, 2006
4. Antti Laaksonen, “Competitive Programmer’s Handbook”, Hand book, 2018
5. Steven Halim and Felix Halim, “Competitive Programming 3: The Lower Bounds of Programming Contests”, Handbook for ACM ICPC

Websites:

1. https://www.geeksforgeeks.org/dsa/sparse-table/
2. https://codeforces.com/blog/entry/139609